

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import GBTClassifier
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

print("1. Εκκίνηση αυτόνομης SparkSession για GBT (Διορθωμένο)...")
spark = SparkSession.builder \
    .appName("GBT_SMOTE_Corrected_GridSearch") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .getOrCreate()

print("2. Φόρτωση Δεδομένων (Απευθείας από το ΔΙΟΡΘΩΜΕΝΟ SMOTE Gold dataset)...")
train_gold = spark.read.parquet("../data/train_gold_corrected.parquet")
test_gold = spark.read.parquet("../data/test_gold_corrected.parquet")

train_gold = train_gold.withColumn("stroke",
train_gold["stroke"].cast(DoubleType()))
test_gold = test_gold.withColumn("stroke",
test_gold["stroke"].cast(DoubleType()))
train_gold = train_gold.withColumn("cluster",
F.col("cluster").cast(DoubleType()))
test_gold = test_gold.withColumn("cluster", F.col("cluster").cast(DoubleType()))

print("3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...")
assembler = VectorAssembler(inputCols=["features", "cluster"],
outputCol="augmented_features")

train_augmented =
assembler.transform(train_gold).drop("features").withColumnRenamed("augmented_features",
"features")
test_augmented =
assembler.transform(test_gold).drop("features").withColumnRenamed("augmented_features",
"features")

train_augmented.cache()
test_augmented.cache()

print("4. Ορισμός GBT και ΒΕΛΤΙΣΤΟΠΟΙΗΜΕΝΟΥ Πλέγματος Παραμέτρων (Σύμφωνα με το
Αρχικό Πλάνο)...")
gbt_base = GBTClassifier(featuresCol="features", labelCol="stroke",
seed=22390225)

# Επιστροφή στις ΔΙΚΕΣ ΣΟΥ ρυθμίσεις που προστάτευαν το Recall,
# αλλά αφήνουμε το Grid να διαλέξει το βέλτιστο micro-tuning.

```

```

paramGrid = (ParamGridBuilder()
              .addGrid(gbt_base.maxDepth, [2, 3])           # Μόνο stumps και
ρηχά δέντρα (Αποφυγή overfitting)
              .addGrid(gbt_base.maxIter, [10, 20, 30, 40])  # Πολύ χαμηλές
επαναλήψεις (Early Stopping)
              .addGrid(gbt_base.stepSize, [0.05, 0.1])      # Συντηρητική
μάθηση
              .build())

# Χρησιμοποιούμε ROC-AUC για να μην κοιτάει το τεχνητό Precision του SMOTE
evaluator = BinaryClassificationEvaluator(
    labelCol="stroke",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

cv = CrossValidator(estimator=gbt_base,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=3,
                    seed=22390225)

print("5. Εκτέλεση Cross-Validation στο SMOTE Dataset με βάση το ROC-AUC...")
cv_model = cv.fit(train_augmented)
best_gbt = cv_model.bestModel

print("\n" + "="*60)
print("[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ GBT ΟΛΟΚΛΗΡΩΘΗΚΕ]")
print(f" -> Βέλτιστο maxDepth: {best_gbt._java_obj.getMaxDepth()}")
print(f" -> Βέλτιστο maxIter (αριθμός δέντρων): {best_gbt._java_obj.getMaxIter()}")
print(f" -> Βέλτιστο stepSize (Learning Rate): {best_gbt._java_obj.getStepSize()}")
print("="*60 + "\n")

print("6. Παραγωγή προβλέψεων στο άγνωστο Test Set...")
gbt_preds = best_gbt.transform(test_augmented)

print("7. Αποθήκευση των τελικών προβλέψεων...")
output_path = "../data/gbt_predictions.parquet"
gbt_preds.select("stroke", "prediction",
"probability").write.mode("overwrite").parquet(output_path)

print(f"=====")
print(f"[ΕΠΙΤΥΧΙΑ] Το διορθωμένο GBT εκπαιδεύτηκε και αποθηκεύτηκε!")
print(f"=====")

spark.stop()

```

1. Εκκίνηση αυτόνομης SparkSession για GBT (Διορθωμένο)...

```

WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/06/12 03:51:59 WARN Utils: Your hostname, cachyos-x8664, resolves to a
loopback address: 127.0.1.1; using 192.168.1.5 instead (on interface enp4s0)
26/06/12 03:51:59 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
26/06/12 03:51:59 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable

2. Φόρτωση Δεδομένων (Απευθείας από το ΔΙΟΡΘΩΜΕΝΟ SMOTE Gold dataset)...
3. Feature Augmentation (Ενσωμάτωση K-Means Cluster)...
4. Ορισμός GBT και ΒΕΛΤΙΣΤΟΠΟΙΗΜΕΝΟΥ Πλέγματος Παραμέτρων (Σύμφωνα με το Αρχικό
Πλάνο)...
5. Εκτέλεση Cross-Validation στο SMOTE Dataset με βάση το ROC-AUC...

```

```

=====
[EΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ GBT ΟΛΟΚΛΗΡΩΘΗΚΕ]

```

```

-> Βέλτιστο maxDepth: 3
-> Βέλτιστο maxIter (αριθμός δέντρων): 40
-> Βέλτιστο stepSize (Learning Rate): 0.1
=====

```

```

6. Παραγωγή προβλέψεων στο άγνωστο Test Set...
7. Αποθήκευση των τελικών προβλέψεων...

```

```

=====
[ΕΠΙΤΥΧΙΑ] Το διορθωμένο GBT εκπαιδεύτηκε και αποθηκεύτηκε!
=====

```